V1 Architecture, Design and Threat Modeling

Control Objective

Security architecture has almost become a lost art in many organizations. The days of the enterprise architect have passed in the age of DevSecOps. The application security field must catch up and adopt agile security principles while re-introducing leading security architecture principles to software practitioners. Architecture is not an implementation, but a way of thinking about a problem that has potentially many different answers, and no one single "correct" answer. All too often, security is seen as inflexible and demanding that developers fix code in a particular way, when the developers may know a much better way to solve the problem. There is no single, simple solution for architecture, and to pretend otherwise is a disservice to the software engineering field.

A specific implementation of a web application is likely to be revised continuously throughout its lifetime, but the overall architecture will likely rarely change but evolve slowly. Security architecture is identical - we need authentication today, we will require authentication tomorrow, and we will need it five years from now. If we make sound decisions today, we can save a lot of effort, time, and money if we select and re-use architecturally compliant solutions. For example, a decade ago, multi-factor authentication was rarely implemented.

If developers had invested in a single, secure identity provider model, such as SAML federated identity, the identity provider could be updated to incorporate new requirements such as NIST 800-63 compliance, while not changing the interfaces of the original application. If many applications shared the same security architecture and thus that same component, they all benefit from this upgrade at once. However, SAML will not always remain as the best or most suitable authentication solution - it might need to be swapped out for other solutions as requirements change. Changes like this are either complicated, so costly as to necessitate a complete re-write, or outright impossible without security architecture.

In this chapter, the ASVS covers off the primary aspects of any sound security architecture: availability, confidentiality, processing integrity, non-repudiation, and privacy. Each of these security principles must be built in and be innate to all applications. It is critical to "shift left", starting with developer enablement with secure coding checklists, mentoring and training, coding and testing, building, deployment, configuration, and operations, and finishing with follow up independent testing to assure that all of the security controls are present and functional. The last step used to be everything we did as an industry, but that is no longer sufficient when developers push code into production tens or hundreds of times a day. Application security professionals must keep up with agile techniques, which means adopting developer tools, learning to code, and working with developers rather than criticizing the project months after everyone else has moved on.

V1.1 Secure Software Development Lifecycle

#	Description	L1	L2	L3	CWE
1.1.1	Verify the use of a secure software development lifecycle that addresses security in all stages of development. (C1)		✓	✓	
1.1.2	Verify the use of threat modeling for every design change or sprint planning to identify threats, plan for countermeasures, facilitate appropriate risk responses, and guide security testing.		✓	✓	1053
1.1.3	Verify that all user stories and features contain functional security constraints, such as "As a user, I should be able to view and edit my profile. I should not be able to view or edit anyone else's profile"		✓	✓	1110
1.1.4	Verify documentation and justification of all the application's trust boundaries, components, and significant data flows.		✓	✓	1059
1.1.5	Verify definition and security analysis of the application's high-level architecture and all connected remote services. (C1)		✓	✓	1059

#	Description	L1	L2	L3	CWE
1.1.6	Verify implementation of centralized, simple (economy of design), vetted, secure, and reusable security controls to avoid duplicate, missing, ineffective, or insecure controls. (C10)		✓	✓	637
1.1.7	Verify availability of a secure coding checklist, security requirements, guideline, or policy to all developers and testers.		✓	✓	637

V1.2 Authentication Architecture

When designing authentication, it doesn't matter if you have strong hardware enabled multi-factor authentication if an attacker can reset an account by calling a call center and answering commonly known questions. When proving identity, all authentication pathways must have the same strength.

#	Description	L1	L2	L3	CWE
1.2.1	Verify the use of unique or special low-privilege operating system accounts for all application components, services, and servers. (C3)		✓	✓	250
1.2.2	Verify that communications between application components, including APIs, middleware and data layers, are authenticated. Components should have the least necessary privileges needed. (C3)		✓	✓	306
1.2.3	Verify that the application uses a single vetted authentication mechanism that is known to be secure, can be extended to include strong authentication, and has sufficient logging and monitoring to detect account abuse or breaches.		✓	✓	306
1.2.4	Verify that all authentication pathways and identity management APIs implement consistent authentication security control strength, such that there are no weaker alternatives per the risk of the application.		✓	✓	306

V1.3 Session Management Architecture

This is a placeholder for future architectural requirements.

V1.4 Access Control Architecture

#	Description	L1	L2	L3	CWE
1.4.1	Verify that trusted enforcement points, such as access control gateways, servers, and serverless functions, enforce access controls. Never enforce access controls on the client.		✓	✓	602
1.4.2	[DELETED, NOT ACTIONABLE]				
1.4.3	[DELETED, DUPLICATE OF 4.1.3]				
1.4.4	Verify the application uses a single and well-vetted access control mechanism for accessing protected data and resources. All requests must pass through this single mechanism to avoid copy and paste or insecure alternative paths. (C7)		✓	✓	284
1.4.5	Verify that attribute or feature-based access control is used whereby the code checks the user's authorization for a feature/data item rather than just their role. Permissions should still be allocated using roles. (C7)		✓	✓	275

V1.5 Input and Output Architecture

In 4.0, we have moved away from the term "server-side" as a loaded trust boundary term. The trust boundary is still concerning - making decisions on untrusted browsers or client devices is bypassable. However, in mainstream architectural deployments today, the trust enforcement point has dramatically changed. Therefore, where the term "trusted service layer" is used in the ASVS, we mean any trusted enforcement point, regardless of location, such as a microservice, serverless API, server-side, a trusted API on a client device that has secure boot, partner or external APIs, and so on.

The "untrusted client" term here refers to client-side technologies that render the presentation layer, commonly refered to as 'front-end' technologies. The term "serialization" here not only refers to sending data over the wire like an array of values or taking and reading a JSON structure, but also passing complex objects which can contain logic.

#	Description	L1	L2	L3	CWE
1.5.1	Verify that input and output requirements clearly define how to handle and process data based on type, content, and applicable laws, regulations, and other policy compliance.		✓	✓	1029
1.5.2	Verify that serialization is not used when communicating with untrusted clients. If this is not possible, ensure that adequate integrity controls (and possibly encryption if sensitive data is sent) are enforced to prevent deserialization attacks including object injection.		✓	✓	502
1.5.3	Verify that input validation is enforced on a trusted service layer. (C5)		✓	✓	602
1.5.4	Verify that output encoding occurs close to or by the interpreter for which it is intended. (C4)		✓	✓	116

V1.6 Cryptographic Architecture

Applications need to be designed with strong cryptographic architecture to protect data assets as per their classification. Encrypting everything is wasteful, not encrypting anything is legally negligent. A balance must be struck, usually during architectural or high level design, design sprints or architectural spikes. Designing cryptography as you go or retrofitting it will inevitably cost much more to implement securely than simply building it in from the start.

Architectural requirements are intrinsic to the entire code base, and thus difficult to unit or integrate test. Architectural requirements require consideration in coding standards, throughout the coding phase, and should be reviewed during security architecture, peer or code reviews, or retrospectives.

#	Description	L1	L2	L3	CWE
1.6.1	Verify that there is an explicit policy for management of cryptographic keys and that a cryptographic key lifecycle follows a key management standard such as NIST SP 800-57.		✓	✓	320
1.6.2	Verify that consumers of cryptographic services protect key material and other secrets by using key vaults or API based alternatives.		✓	✓	320
1.6.3	Verify that all keys and passwords are replaceable and are part of a well-defined process to re-encrypt sensitive data.		✓	✓	320
1.6.4	Verify that the architecture treats client-side secrets--such as symmetric keys, passwords, or API tokens--as insecure and never uses them to protect or access sensitive data.		✓	✓	320

V1.7 Errors, Logging and Auditing Architecture

#	Description	L1	L2	L3	CWE
1.7.1	Verify that a common logging format and approach is used across the system. (C9)		✓	✓	1009
1.7.2	Verify that logs are securely transmitted to a preferably remote system for analysis, detection, alerting, and escalation. (C9)		✓	✓	

V1.8 Data Protection and Privacy Architecture

#	Description	L1	L2	L3	CWE
1.8.1	Verify that all sensitive data is identified and classified into protection levels.		✓	✓	
1.8.2	Verify that all protection levels have an associated set of protection requirements, such as encryption requirements, integrity requirements, retention, privacy and other confidentiality requirements, and that these are applied in the architecture.		✓	✓	

V1.9 Communications Architecture

#	Description	L1	L2	L3	CWE
1.9.1	Verify the application encrypts communications between components, particularly when these components are in different containers, systems, sites, or cloud providers. (C3)		✓	✓	319
1.9.2	Verify that application components verify the authenticity of each side in a communication link to prevent person-in-the-middle attacks. For example, application components should validate TLS certificates and chains.		✓	✓	295

V1.10 Malicious Software Architecture

#	Description	L1	L2	L3	CWE
1.10.1	Verify that a source code control system is in use, with procedures to ensure that check-ins are accompanied by issues or change tickets. The source code control system should have access control and identifiable users to allow traceability of any changes.		✓	✓	284

V1.11 Business Logic Architecture

#	Description	L1	L2	L3	CWE
1.11.1	Verify the definition and documentation of all application components in terms of the business or security functions they provide.		✓	✓	1059
1.11.2	Verify that all high-value business logic flows, including authentication, session management and access control, do not share unsynchronized state.		✓	✓	362
1.11.3	Verify that all high-value business logic flows, including authentication, session management and access control are thread safe and resistant to time-of-check and time-of-use race conditions.			✓	367

V1.12 Secure File Upload Architecture

#	Description	L1	L2	L3	CWE
1.12.1	[DELETED, DUPLICATE OF 12.4.1]				
1.12.2	Verify that user-uploaded files - if required to be displayed or downloaded from the application - are served by either octet stream downloads, or from an unrelated domain, such as a cloud file storage bucket. Implement a suitable Content Security Policy (CSP) to reduce the risk from XSS vectors or other attacks from the uploaded file.		✓	✓	646

V1.13 API Architecture

This is a placeholder for future architectural requirements.

V1.14 Configuration Architecture

#	Description	L1	L2	L3	CWE
1.14.1	Verify the segregation of components of differing trust levels through well-defined security controls, firewall rules, API gateways, reverse proxies, cloud-based security groups, or similar mechanisms.		✓	✓	923
1.14.2	Verify that binary signatures, trusted connections, and verified endpoints are used to deploy binaries to remote devices.		✓	✓	494
1.14.3	Verify that the build pipeline warns of out-of-date or insecure components and takes appropriate actions.		✓	✓	1104
1.14.4	Verify that the build pipeline contains a build step to automatically build and verify the secure deployment of the application, particularly if the application infrastructure is software defined, such as cloud environment build scripts.		✓	✓	
1.14.5	Verify that application deployments adequately sandbox, containerize and/or isolate at the network level to delay and deter attackers from attacking other applications, especially when they are performing sensitive or dangerous actions such as deserialization. (C5)		✓	✓	265
1.14.6	Verify the application does not use unsupported, insecure, or deprecated client-side technologies such as NSAPI plugins, Flash, Shockwave, ActiveX, Silverlight, NACL, or client-side Java applets.		✓	✓	477

References

For more information, see also:

- OWASP Threat Modeling Cheat Sheet
- OWASP Attack Surface Analysis Cheat Sheet
- OWASP Threat modeling
- OWASP Software Assurance Maturity Model Project
- Microsoft SDL
- NIST SP 800-57